

202 ➤ 函数是一个命名的代码块，我们通过调用函数执行相应的代码。函数可以有 0 个或多个参数，而且（通常）会产生一个结果。可以重载函数，也就是说，同一个名字可以对应几个不同的函数。



6.1 函数基础

一个典型的函数（function）定义包括以下部分：返回类型（return type）、函数名字、由 0 个或多个形参（parameter）组成的列表以及函数体。其中，形参以逗号隔开，形参的列表位于一对圆括号之内。函数执行的操作在语句块（参见 5.1 节，第 155 页）中说明，该语句块称为函数体（function body）。

我们通过调用运算符（call operator）来执行函数。调用运算符的形式是一对圆括号，它作用于一个表达式，该表达式是函数或者指向函数的指针；圆括号之内是一个用逗号隔开的实参（argument）列表，我们用实参初始化函数的形参。调用表达式的类型就是函数的返回类型。

编写函数

举个例子，我们准备编写一个求数的阶乘的程序。 n 的阶乘是从 1 到 n 所有数字的乘积，例如 5 的阶乘是 120。

$1 * 2 * 3 * 4 * 5 = 120$

程序如下所示：

```
// val 的阶乘是 val*(val - 1)*(val - 2) ...*((val - (val - 1))* 1)
int fact(int val)
{
    int ret = 1;           // 局部变量，用于保存计算结果
    while (val > 1)
        ret *= val--;      // 把 ret 和 val 的乘积赋给 ret，然后将 val 减 1
    return ret;           // 返回结果
}
```

函数的名字是 fact，它作用于一个整型参数，返回一个整型值。在 while 循环内部，在每次迭代时用后置递减运算符（参见 4.5 节，第 131 页）将 val 的值减 1。return 语句负责结束 fact 并返回 ret 的值。

调用函数

要调用 fact 函数，必须提供一个整数值，调用得到的结果也是一个整数：

```
int main()
{
    int j = fact(5);       // j 等于 120，即 fact(5) 的结果
    cout << "5! is " << j << endl;
    return 0;
}
```

203 ➤ 函数的调用完成两项工作：一是用实参初始化函数对应的形参，二是将控制权转移给被调用函数。此时，主调函数（calling function）的执行被暂时中断，被调函数（called function）开始执行。

执行函数的第一步是（隐式地）定义并初始化它的形参。因此，当调用 `fact` 函数时，首先创建一个名为 `val` 的 `int` 变量，然后将它初始化为调用时所用的实参 5。

当遇到一条 `return` 语句时函数结束执行过程。和函数调用一样，`return` 语句也完成两项工作：一是返回 `return` 语句中的值（如果有的话），二是将控制权从被调函数转移回主调函数。函数的返回值用于初始化调用表达式的结果，之后继续完成调用所在的表达式的剩余部分。因此，我们对 `fact` 函数的调用等价于如下形式：

```
int val = 5;           // 用字面值 5 初始化 val
int ret = 1;          // fact 函数体内的代码
while (val > 1)
    ret *= val--;
int j = ret;           // 用 ret 的副本初始化 j
```

形参和实参

实参是形参的初始值。第一个实参初始化第一个形参，第二个实参初始化第二个形参，以此类推。尽管实参与形参存在对应关系，但是并没有规定实参的求值顺序（参见 4.1.3 节，第 123 页）。编译器能以任意可行的顺序对实参求值。

实参的类型必须与对应的形参类型匹配，这一点与之前的规则是一致的，我们知道在初始化过程中初始值的类型也必须与初始化对象的类型匹配。函数有几个形参，我们就必须提供相同数量的实参。因为函数的调用规定实参数量应与形参数量一致，所以形参一定会被初始化。

在上面的例子中，`fact` 函数只有一个 `int` 类型的形参，所以每次我们调用它的时候，都必须提供一个能转换（参见 4.11 节，第 141 页）成 `int` 的实参：

```
fact("hello");        // 错误：实参类型不正确
fact();               // 错误：实参数量不足
fact(42, 10, 0);      // 错误：实参数量过多
fact(3.14);           // 正确：该实参能转换成 int 类型
```

因为不能将 `const char*` 转换成 `int`，所以第一个调用失败。第二个和第三个调用也会失败，不过错误的原因与第一个不同，它们是因为传入的实参数量不对。要想调用 `fact` 函数只能使用一个实参，只要实参数量不是一个，调用都将失败。最后一个调用是合法的，因为 `double` 可以转换成 `int`。执行调用时，实参隐式地转换成 `int` 类型（截去小数部分），调用等价于

```
fact(3);
```

函数的形参列表

◀ 204

函数的形参列表可以为空，但是不能省略。要想定义一个不带形参的函数，最常用的办法是书写一个空的形参列表。不过为了与 C 语言兼容，也可以使用关键字 `void` 表示函数没有形参：

```
void f1(){ /* ...*/ }      // 隐式地定义空形参列表
void f2(void){ /* ...*/ }  // 显式地定义空形参列表
```

形参列表中的形参通常用逗号隔开，其中每个形参都是含有一个声明符的声明。即使两个形参的类型一样，也必须把两个类型都写出来：

```
int f3(int v1, v2) { /* ...*/ }      // 错误
int f4(int v1, int v2) { /* ...*/ }  // 正确
```

任意两个形参都不能同名，而且函数最外层作用域中的局部变量也不能使用与函数形参一样的名字。

形参名是可选的，但是由于我们无法使用未命名的形参，所以形参一般都应该有个名字。偶尔，函数确实有个别形参不会被用到，则此类形参通常不命名以表示在函数体内不会使用它。不管怎样，是否设置未命名的形参并不影响调用时提供的实参数量。即使某个形参不被函数使用，也必须为它提供一个实参。

函数返回类型

大多数类型都能用作函数的返回类型。一种特殊的返回类型是 `void`，它表示函数不返回任何值。函数的返回类型不能是数组（参见 3.5 节，第 101 页）类型或函数类型，但可以是指向数组或函数的指针。我们将在 6.3.3 节（第 205 页）介绍如何定义一种特殊的函数，它的返回值是数组的指针（或引用），在 6.7 节（第 221 页）将介绍如何返回指向函数的指针。

6.1 节练习

练习 6.1：实参和形参的区别是什么？

练习 6.2：请指出下列函数哪个有错误，为什么？应该如何修改这些错误呢？

```
(a) int f() {  
    string s;  
    //...  
    return s;  
}  
  
(b) f2(int i) { /* ...*/ }  
(c) int calc(int v1, int v1) /* ...*/ }  
(d) double square(double x) return x * x;
```

练习 6.3：编写你自己的 `fact` 函数，上机检查是否正确。

练习 6.4：编写一个与用户交互的函数，要求用户输入一个数字，计算生成该数字的阶乘。在 `main` 函数中调用该函数。

练习 6.5：编写一个函数输出其实参的绝对值。



6.1.1 局部对象

在 C++ 语言中，名字有作用域（参见 2.2.4 节，第 43 页），对象有生命周期（lifetime）。理解这两个概念非常重要。

- 名字的作用域是程序文本的一部分，名字在其中可见。
- 对象的生命周期是程序执行过程中该对象存在的一段时间。

如我们所知，函数体是一个语句块。块构成一个新的作用域，我们可以在其中定义变量。形参和函数体内部定义的变量统称为**局部变量**（local variable）。它们对函数而言是“局部”的，仅在函数的作用域内可见，同时局部变量还会**隐藏**（hide）在外层作用域中同名的其他所有声明中。

在所有函数体之外定义的对象存在于程序的整个执行过程中。此类对象在程序启动时被创建，直到程序结束才会销毁。局部变量的生命周期依赖于定义的方式。

自动对象

对于普通局部变量对应的对象来说，当函数的控制路径经过变量定义语句时创建该对象，当到达定义所在的块末尾时销毁它。我们把只存在于块执行期间的对象称为**自动对象**（automatic object）。当块的执行结束后，块中创建的自动对象的值就变成未定义的了。

形参是一种自动对象。函数开始时为形参申请存储空间，因为形参定义在函数体作用域之内，所以一旦函数终止，形参也就被销毁。

我们用传递给函数的实参初始化形参对应的自动对象。对于局部变量对应的自动对象来说，则分为两种情况：如果变量定义本身含有初始值，就用这个初始值进行初始化；否则，如果变量定义本身不含初始值，执行默认初始化（参见 2.2.1 节，第 40 页）。这意味着内置类型的未初始化局部变量将产生未定义的值。

局部静态对象

某些时候，有必要令局部变量的生命周期贯穿函数调用及之后的时间。可以将局部变量定义成 `static` 类型从而获得这样的对象。**局部静态对象**（local static object）在程序的执行路径第一次经过对象定义语句时初始化，并且直到程序终止才被销毁，在此期间即使对象所在的函数结束执行也不会对它有影响。

◀ 206

举个例子，下面的函数统计它自己被调用了多少次，这样的函数也许没什么实际意义，但是足够说明问题：

```
size_t count_calls()
{
    static size_t ctr = 0; // 调用结束后，这个值仍然有效
    return ++ctr;
}

int main()
{
    for (size_t i = 0; i != 10; ++i)
        cout << count_calls() << endl;
    return 0;
}
```

这段程序将输出从 1 到 10（包括 10 在内）的数字。

在控制流第一次经过 `ctr` 的定义之前，`ctr` 被创建并初始化为 0。每次调用将 `ctr` 加 1 并返回新值。每次执行 `count_calls` 函数时，变量 `ctr` 的值都已经存在并且等于函数上一次退出时 `ctr` 的值。因此，第二次调用时 `ctr` 的值是 1，第三次调用时 `ctr` 的值是 2，以此类推。

如果局部静态变量没有显式的初始值，它将执行值初始化（参见 3.3.1 节，第 88 页），内置类型的局部静态变量初始化为 0。

6.1.1 节练习

练习 6.6：说明形参、局部变量以及局部静态变量的区别。编写一个函数，同时用到这三种形式。

练习 6.7：编写一个函数，当它第一次被调用时返回 0，以后每次被调用返回值加 1。



6.1.2 函数声明

和其他名字一样，函数的名字也必须在使用之前声明。类似于变量（参见 2.2.2 节，第 41 页），函数只能定义一次，但可以声明多次。唯一的例外是如 15.3 节（第 535 页）将要介绍的，如果一个函数永远也不会被我们用到，那么它可以只有声明没有定义。

函数的声明和函数的定义非常类似，唯一的区别是函数声明无须函数体，用一个分号替代即可。

因为函数的声明不包含函数体，所以也就无须形参的名字。事实上，在函数的声明中经常省略形参的名字。尽管如此，写上形参的名字还是有用处的，它可以帮助使用者更好地理解函数的功能：

207

```
// 我们选择 beg 和 end 作为形参的名字以表示这两个迭代器划定了输出值的范围
void print(vector<int>::const_iterator beg,
          vector<int>::const_iterator end);
```

函数的三要素（返回类型、函数名、形参类型）描述了函数的接口，说明了调用该函数所需的全部信息。函数声明也称作**函数原型**（function prototype）。

在头文件中进行函数声明

回忆之前所学的知识，我们建议变量在头文件（参见 2.6.3 节，第 68 页）中声明，在源文件中定义。与之类似，函数也应该在头文件中声明而在源文件中定义。

看起来把函数的声明直接放在使用该函数的源文件中是合法的，也比较容易被人接受；但是这么做可能会很烦琐而且容易出错。相反，如果把函数声明放在头文件中，就能确保同一函数的所有声明保持一致。而且一旦我们想改变函数的接口，只需改变一条声明即可。

定义函数的源文件应该把含有函数声明的头文件包含进来，编译器负责验证函数的定义和声明是否匹配。

Best
Practices

含有函数声明的头文件应该被包含到定义函数的源文件中。

6.1.2 节练习

练习 6.8：编写一个名为 Chapter6.h 的头文件，令其包含 6.1 节练习（第 184 页）中的函数声明。



6.1.3 分离式编译

随着程序越来越复杂，我们希望把程序的各个部分分别存储在不同文件中。例如，可以把 6.1 节练习（第 184 页）的函数存在一个文件里，把使用这些函数的代码存在其他源文件中。为了允许编写程序时按照逻辑关系将其划分开来，C++ 语言支持所谓的**分离式编译**（separate compilation）。分离式编译允许我们把程序分割到几个文件中去，每个文件独立编译。

编译和链接多个源文件

举个例子，假设 fact 函数的定义位于一个名为 fact.cc 的文件中，它的声明位于

名为 Chapter6.h 的头文件中。显然与其他所有用到 fact 函数的文件一样, fact.cc 应该包含 Chapter6.h 头文件。另外,我们在名为 factMain.cc 的文件中创建 main 函数,main 函数将调用 fact 函数。要生成可执行文件 (executable file), 必须告诉编译器我们用到的代码在哪里。对于上述几个文件来说,编译的过程如下所示:

```
$ CC factMain.cc fact.cc # generates factMain.exe or a.out
$ CC factMain.cc fact.cc -o main # generates main or main.exe
```

其中,CC 是编译器的名字,\$是系统提示符,#后面是命令行下的注释语句。接下来运行可执行文件,就会执行我们定义的 main 函数。

如果我们修改了其中一个源文件,那么只需重新编译那个改动了的文件。大多数编译器提供了分离式编译每个文件的机制,这一过程通常会产生一个后缀名是 .obj (Windows) 或 .o (UNIX) 的文件,后缀名的含义是该文件包含对象代码 (object code)。

接下来编译器负责把对象文件链接在一起形成可执行文件。在我们的系统中,编译的过程如下所示:

```
$ CC -c factMain.cc # generates factMain.o
$ CC -c fact.cc # generates fact.o
$ CC factMain.o fact.o # generates factMain.exe or a.out
$ CC factMain.o fact.o -o main # generates main or main.exe
```

你可以仔细阅读编译器的用户手册,弄清楚由多个文件组成的程序是如何编译并执行的。

6.1.3 节练习

练习 6.9: 编写你自己的 fact.cc 和 factMain.cc,这两个文件都应该包含上一小节的练习中编写的 Chapter6.h 头文件。通过这些文件,理解你的编译器是如何支持分离式编译的。

6.2 参数传递



如前所述,每次调用函数时都会重新创建它的形参,并用传入的实参对形参进行初始化。



形参初始化的机理与变量初始化一样。

和其他变量一样,形参的类型决定了形参和实参交互的方式。如果形参是引用类型(参见 2.3.1 节,第 45 页),它将绑定到对应的实参上;否则,将实参的值拷贝后赋给形参。

当形参是引用类型时,我们说它对应的实参被**引用传递** (passed by reference) 或者函数被**传引用调用** (called by reference)。和其他引用一样,引用形参也是它绑定的对象的别名;也就是说,引用形参是它对应的实参的别名。

当实参的值被拷贝给形参时,形参和实参是两个相互独立的对象。我们说这样的实参被**值传递** (passed by value) 或者函数被**传值调用** (called by value)。

6.2.1 传值参数



当初始化一个非引用类型的变量时,初始值被拷贝给变量。此时,对变量的改动不会